Global Capacity Orchestrator

One API. Every Accelerator. Any Region.

Multi-region orchestration for accelerated compute on AWS — NVIDIA GPUs, AWS Trainium, AWS Inferentia, and CPU — with capacity-aware scheduling, spot fallback, and autoscaling multi-region inference, all from one IAM-authenticated API and CLI.

Introduction & Capabilities Overview

github.com/awslabs/global-capacity-orchestrator-on-aws

aws

Agenda

What we'll cover

01 **The opportunity**

GPU scarcity, multi-region pain, and what GCO is

02 **Architecture**

Global routing, the regional stack, and security in depth

03 **Running workloads**

One-command deploys, batch schedulers, and global inference

04 **The Platform**

Storage, analytics, the MCP server, and Mission

05 **Foundation & demo**

Engineering quality, operations, and a live demo

01

The Opportunity

GPU scarcity, multi-region pain, and what GCO is

The problem: finding and using GPU capacity

What customers keep telling us

“We can't get GPU capacity in our primary region.”

“Managing multi-region infrastructure is a nightmare.”

“Our teams spend more time on infra than on ML.”

“How do I serve multi-region inference with automatic failover?”

“How do I scale my workload when capacity runs out?”

Demand is outpacing supply.

There is real capacity out there that customers can't easily find or use.

What if you always knew where capacity was — and could use it instantly, in any region, through one API?

What is Global Capacity Orchestrator?

An AI/ML platform in a box — the complete, MIT-licensed source

- Chain together as many AWS Regions as you want and use the compute for training and inference.
- Infrastructure as pure-Python AWS CDK — one config file (cdk.json), one command to deploy.
- **API + CLI:** A documented REST API with IAM (SigV4) authentication, plus a CLI that makes it easy.
- **MCP:** Built-in MCP server with 120+ tools to drive everything with natural language.
- **Plus:** Integrated SageMaker analytics environment and an autonomous, goal-driven Mission harness.
- **Every accelerator:** Runs NVIDIA GPUs, AWS Trainium, AWS Inferentia, and CPU (amd64 + arm64 / Graviton).

One API. Every Accelerator. Any Region.

One API

A single IAM-authenticated REST API and CLI — no per-cluster kubeconfig, no bespoke glue.

Every Accelerator

NVIDIA GPU (g4dn/g5/g5g), AWS Trainium, AWS Inferentia, and CPU on amd64 + Graviton.

Any Region

EKS Auto Mode clusters across Regions, wired by Global Accelerator with automatic failover.

Scale potential: up to ~27M concurrent GPUs and tens of thousands of submissions/sec across 34 commercial Regions (theoretical maximum — real limits depend on live capacity).

02

Architecture

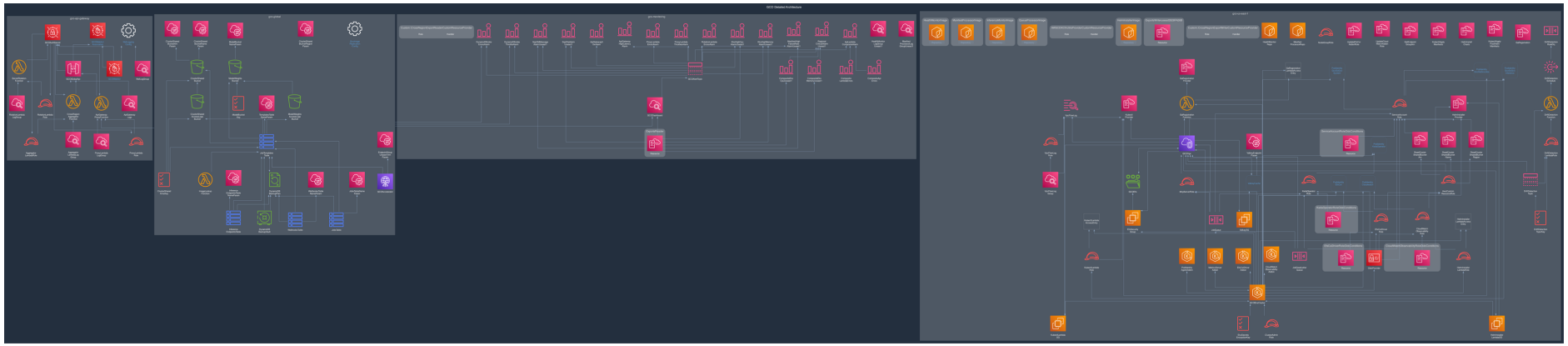
Global routing, the regional stack, and security in depth

aws

How GCO fits together

Four CDK stacks, replicated and wired across Regions

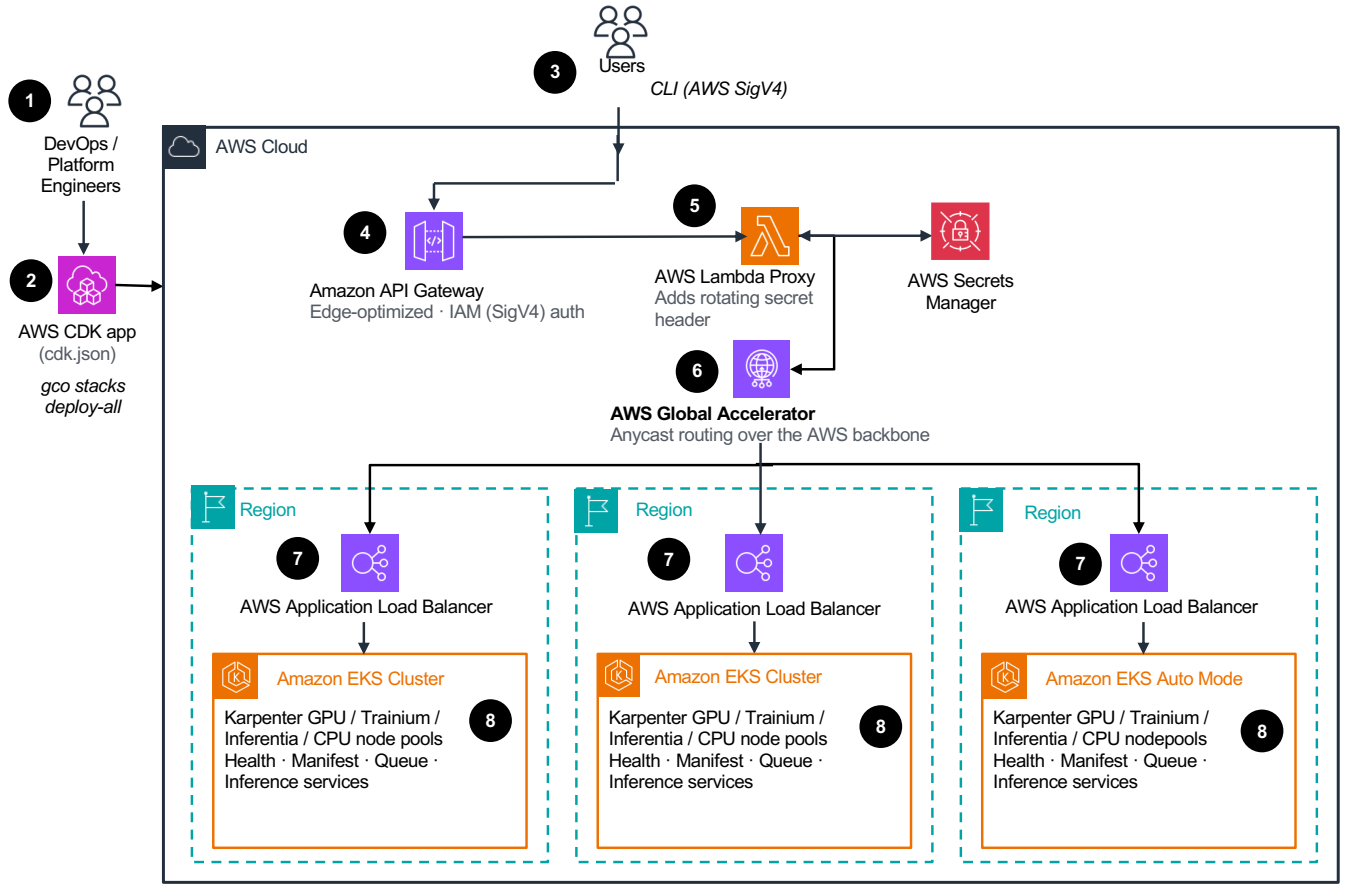
- **Global stack — Global Accelerator:** Routes every request over the AWS backbone to the nearest healthy Region; automatic failover.
- **API Gateway stack:** Edge-optimized REST API with IAM (SigV4) auth, AWS WAF, and a Lambda proxy that injects a rotating secret.
- **Regional stack:** An EKS Auto Mode cluster per Region — ALB, SQS, EFS, nodepools, and platform pods. Replicated in every Region you choose.
- **Monitoring stack:** Cross-Region CloudWatch dashboards, alarms, and SNS alerts from a single pane of glass.
- **Always accurate:** Diagrams are auto-generated from the CDK with AWS PDK — run diagrams/generate.py anytime, they never drift from the code.



https://github.com/awslabs/global-capacity-orchestrator-on-aws/tree/main/diagrams

Guidance for Global Capacity Orchestration of Accelerated Compute on AWS

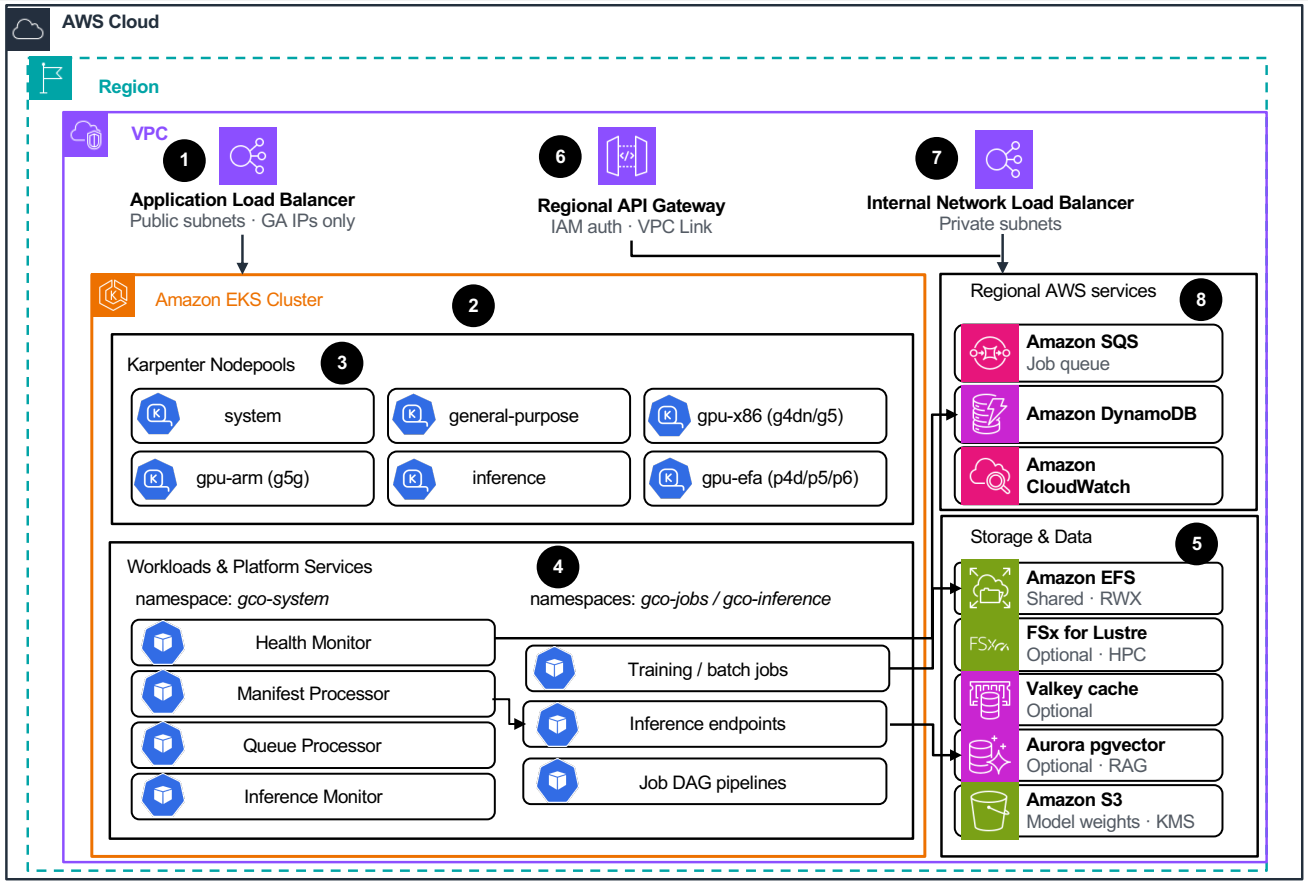
How it is deployed and how requests route: one AWS CDK app stands up every region; one authenticated API/CLI places workloads with failover.



1. **DevOps / Platform Engineers** author and deploy one AWS CDK app; regions are configured in the **cdk.json**.
2. The command `gco stacks deploy-all` is used to provision identical Amazon CloudFormation stacks in every configured AWS Region (auto-bootstrap included).
3. Users submit Kubernetes manifests through one **REST API or CLI** signed with AWS SigV4 (no per-cluster `kubeconfig` configuration is necessary)
4. Amazon API Gateway (edge-optimized) validates IAM credentials at Amazon CloudFront edges and routes the request to the AWS Lambda proxy .
5. The **AWS Lambda** proxy injects a rotating secret from AWS Secrets Manager into user request before forwarding it to AWS Global Accelerator
6. AWS Global Accelerator routes each request over the AWS backbone to the nearest healthy AWS region, with automatic failover.
7. Per-region deployed AWS Application Load Balancers accept traffic only from Global Accelerator IP addresses and route into respective Amazon Elastic Kubernetes System (EKS) clusters.
8. Each AWS region runs an Amazon EKS Auto Mode cluster that provisions GPU, Amazon Trainium, Amazon Inferentia, and CPU based nodes on demand via node pools configurations

Guidance for Global Capacity Orchestration of Accelerated Compute on AWS

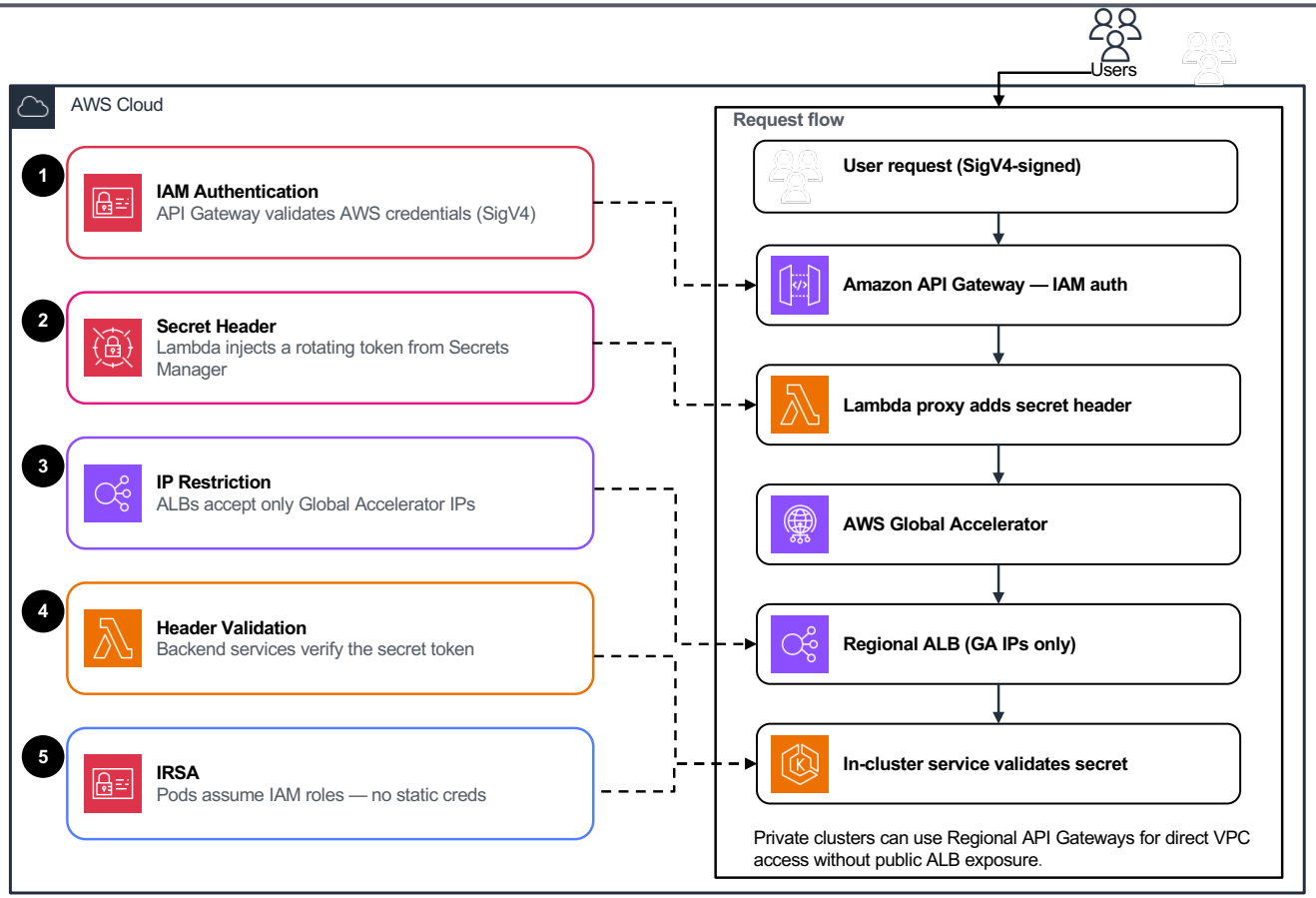
Regional stack: an Amazon EKS Auto Mode cluster in a multi-AZ VPC — ingress, capacity-aware nodepools, platform pods, and persistent storage.



1. An internet-facing Application Load Balancer deployed in Amazon Virtual Private Cloud (VPC) public subnets accepts only from Global Accelerator IPs and routes via Kubernetes Ingress.
2. Amazon EKS Auto Mode cluster runs a managed control plane and auto-provisions nodes — no pre-scaling.
3. **Karpenter nodepools** cover system, general CPU (amd64+arm64), NVIDIA GPU, Trainium/Inferentia, inference, and EFA.
4. Platform pods in **gco-system** namespace operate the workloads in the **gco-jobs namespace**: **Manifest Processor** applies manifests, **Queue Processor** consumes the job queue, **Inference Monitor** reconciles endpoints, and **Health Monitor** tracks EKS cluster health.
5. Workloads persist data to Amazon Elastic File System (EFS) (shared RWX access) and optionally FSx for Lustre; inference workloads uses Valkey cache, Amazon Aurora RDS pgvector vector storage, and Amazon Simple Storage Service (S3) model weights.
6. A **Regional API Gateway** with VPC Link fronts an internal Network Load Balancer for private, in-VPC access to AWS services
7. The internal NLB forwards private traffic to in-cluster services without any public exposure.
8. Regional AWS services — Queue Processor drains Amazon SQS, Inference Monitor reconciles Amazon DynamoDB, and Health Monitor publishes to Amazon CloudWatch.

Guidance for Global Capacity Orchestration of Accelerated Compute on AWS

Security model: a user request traverses the flow on the right; each step is protected by the matching defense layer on the left.



1. **IAM Authentication** — Amazon API Gateway validates users' AWS credentials with SigV4 on every request.
2. **Secret Header** — a Lambda proxy injects a rotating token from AWS Secrets Manager, rotated daily.
3. **IP Restriction** — ALB security groups accept traffic only from AWS Global Accelerator IPs.
4. **Header Validation** — backend services reject any request missing the valid secret token.
5. Amazon IAM roles for Service Accounts (IRSA) — pods assume Amazon IAM roles for AWS access, so no static credentials live in the cluster.
6. Compliance is validated with CDK-nag validation utility against AWS Solutions, HIPAA, NIST 800-53, and PCI DSS standards
7. Default-deny Kubernetes **network policies** plus encryption at rest (Amazon Key Managed Service) and in transit (TLS 1.2+) protect all service communication.

03

Running workloads

One-command deploys, batch schedulers, and global inference

Deploy everything with one command

gco stacks deploy-all -y

- Stands up every Region in cdk.json — CDK bootstrap runs automatically.
- **Fast:** Under 60 minutes even for multi-Region with the --parallel flag.
- **Only prerequisites:** AWS credentials, Git, and a container runtime (Docker / Finch / Podman / Colima).
- **One file:** All configuration lives in one file — cdk.json. Deploys are idempotent: change config, redeploy.

Programmatically recorded: gco stacks deploy-all (source · re-record)

Destroy everything with one command

gco stacks destroy-all -y

- **Clean teardown:** gco stacks destroy-all -y tears it all down — no orphaned resources.

Running batch jobs

Bring your own scheduler — run any combination

Volcano

Gang scheduling, distributed training

Kueue

Resource quotas, fair sharing, priority admission

KubeRay

Distributed compute, HPO, Ray Serve

KEDA

Scale-to-zero, SQS triggers, metric scaling

Slurm (Slinky)

sbatch/srun, HPC migration

YuniKorn

Multi-tenant queues, hierarchical quotas

Four submission methods:

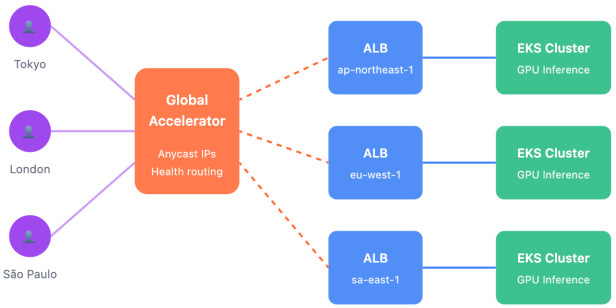
1. Amazon SQS (recommended, KEDA queue processor auto-drains it)
2. API Gateway (SigV4)
3. DynamoDB global queue
4. Direct kubectl.

https://github.com/awslabs/global-capacity-orchestrator-on-aws/blob/main/docs/SCHEDULERS.md

Globally distributed inference

Deploy once, serve everywhere — one CLI command

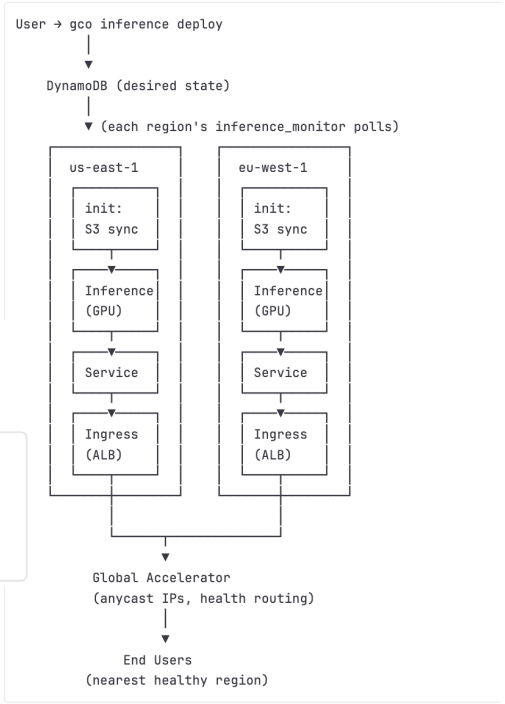
- Deploy endpoints across Regions with a single command: gco inference deploy.
- **Frameworks:** vLLM, TGI, Triton, TorchServe, and SGLang — any containerized server.
- **Reconciliation:** Desired state in DynamoDB; an in-region monitor reconciles Deployments, Services, and Ingress, and self-heals drift.
- **Rollouts:** Canary deployments (weighted A/B), HPA autoscaling, and spot-instance support for cost savings.
- **Weights:** Model weights sync automatically from a central KMS-encrypted S3 bucket into each Region.
- **Routing:** Global Accelerator routes users to the nearest healthy endpoint with automatic failover — end-to-end edge accelerated.



Deploy a vLLM endpoint using the default `facebook/opt-125m` model (small, loads in seconds, good for testing). Omitting `-r` deploys to all regions so Global Accelerator routing works consistently.

```
gco inference deploy vllm-demo \
  -i vllm/vllm-openai:v0.22.0 \
  --gpu-count 1 \
  --replicas 1 \
  --extra-args '--model' --extra-args 'facebook/opt-125m'
```

Inference serving uses a reconciliation pattern similar to Kubernetes controllers:



Read more about how inference works in GCO here:

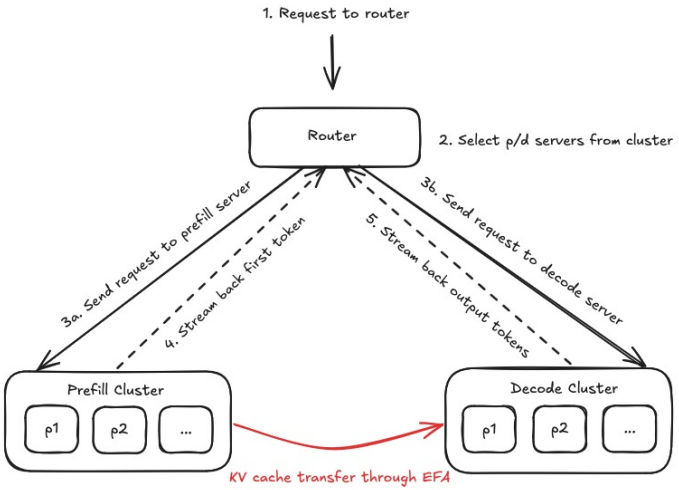
- https://day1inference.com/global-inference-serving/
- https://github.com/awslabs/global-capacity-orchestrator-on-aws/blob/main/docs/INFERENCE.md

Disaggregated inference & KV-cache sharing (Mooncake)

Split prefill from decode, share the KV cache — Mooncake over RDMA on EFA

- **Disaggregated prefill/decode:** Independent prefill (kv_producer) and decode (kv_consumer) pods in an XpYd topology — scale the prompt-bound and generation-bound halves separately, retune live with gco inference set-topology.
- **KV transfer over RDMA:** Prefill streams the KV cache to decode over RoCE/RDMA on the Elastic Fabric Adapter (EFA) — intra-region and zero-copy, wired automatically by the MooncakeConnector.
- **Shared KV store + cold tier:** A per-region MooncakeStore pool adds hash-based prefix-cache reuse across instances; an optional, async S3 cold tier (gco inference populate-kv) warm-starts a region's cache.
- **Three modes, one flag:** --mooncake-mode disaggregated | store | both. An endpoint with no mooncake block behaves exactly as before — fully backward compatible.
- **Right GPUs, automatically:** RDMA role pods are pinned to a curated EFA nodepool of ≥80GB, FP8-capable GPUs (H100/H200/B200 — p5/p6), never undersized 40GB A100s.
- **Same control plane:** Declarative spec in DynamoDB, reconciled in-region; fronted by a PD proxy and Global Accelerator at /inference/{name}/v1 — IAM-authenticated like every GCO endpoint.

Mooncake



https://github.com/kvcache-ai/Mooncake

Capacity finding & auto-routing

Get workloads where they fit — automatically

- **1) Latency + load aware:** Route to the nearest Region, and steer away from clusters already running too much.
  - Resource thresholds (set in cdk.json) drive Global Accelerator health checks — over a threshold, new work routes elsewhere.
- **2) Capacity discovery:** Analyze Spot Placement Scores and live signals to recommend where GPUs are actually available right now.
  - Converse with a Bedrock-powered assistant (default Claude Sonnet) about capacity, grounded in live data.
- **3) One-command inference:** Stand up autoscaling, multi-Region, latency-minimized inference with automatic failover and IAM auth.

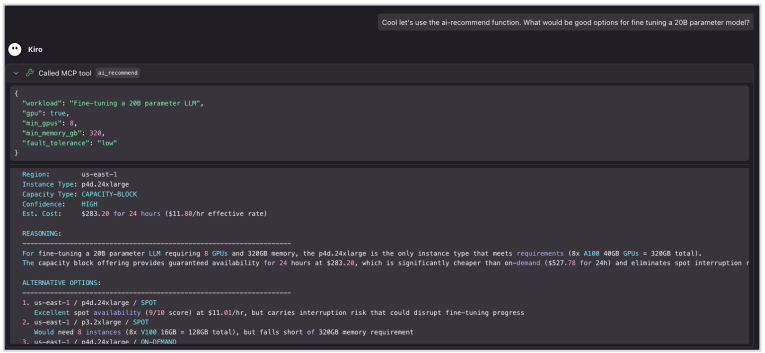
Check GPU availability:

```
gco capacity check --instance-type g5.xlarge --region us-east-1
gco capacity recommend-region --gpu
```

The CLI checks Spot Placement Scores and instance availability across regions to find where GPUs are actually available right now.

Submit with automatic region selection:

```
gco jobs submit-sqs examples/gpu-job.yaml --auto-region
```



AI capacity recommendation via the MCP server

```
"resource_thresholds": {
  "cpu_threshold": 60,
  "memory_threshold": 60,
  "gpu_threshold": 60,
  "pending_pods_threshold": 10,
  "pending_requested_cpu_vcpus": 100,
  "pending_requested_memory_gb": 200,
  "pending_requested_gpus": 8
},
```

Putting it all together

Live demo showing capacity tooling, exercising each scheduler, and deploying and using an inference endpoint

Programmatically generated recording of

- Exercising each scheduler
- Creating an inference endpoint
- Using capacity/cost tooling

Live_demo.gif

04

The platform

Storage, analytics, the MCP server, and Mission

Layered storage options

Auto-provisioned and wired into job pods

Amazon EFS

On by default

Shared elastic file system. gco-shared-storage PVC in gco-jobs/gco-system/default. Ideal for checkpoints, weights, and data that survives pod restarts. ~$0.30/GB-mo.

FSx for Lustre

Opt-in

High-performance parallel file system for large-dataset and distributed training with heavy I/O. S3 data-repository integration.

Valkey cache

Opt-in

Serverless key-value cache for prompt caching and session state.

Aurora pgvector

Opt-in

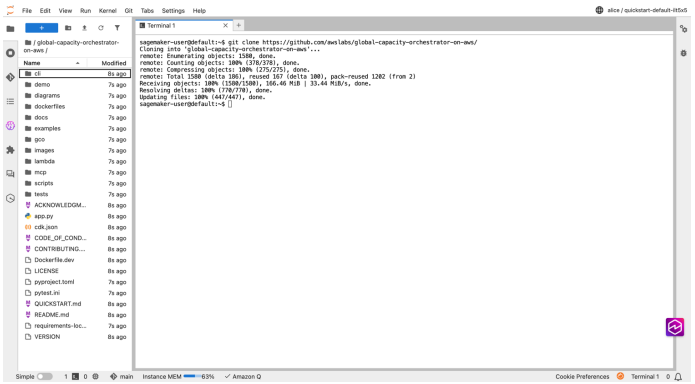
Serverless vector database for RAG, semantic search, and embeddings.

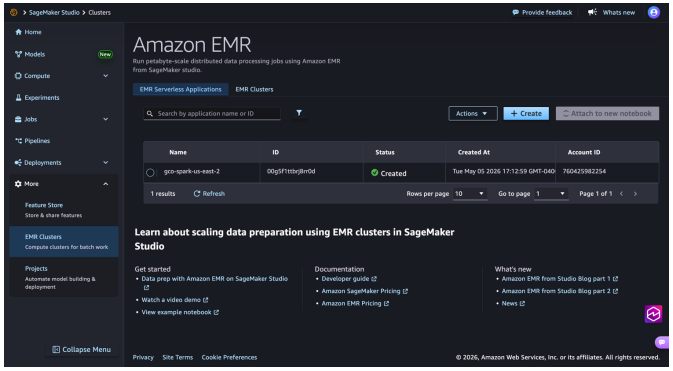
Integrated Analytics Environment

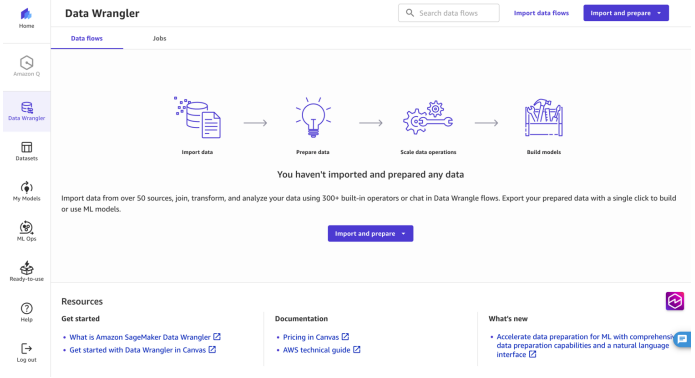
For follow up analysis and multi-tenant collaborative data science and machine learning

- Automated user management with Cognito and GCO CLI
- Encrypted SageMaker environment with KMS key
- EFS for persistent storage
- EMR Serverless cluster for data analysis and Spark jobs
- Shared JupyterLab environment with persistent storage
- Access a shared S3 bucket that all clusters can also read/write to
- Submit to GCO clusters in multiple regions from the environment
- Canvas for no-code ML and data processing
- MLFlow for experiment tracking!
- Deploy with a single command!

- For a complete guide see ANALYTICS.md in the “docs” folder!



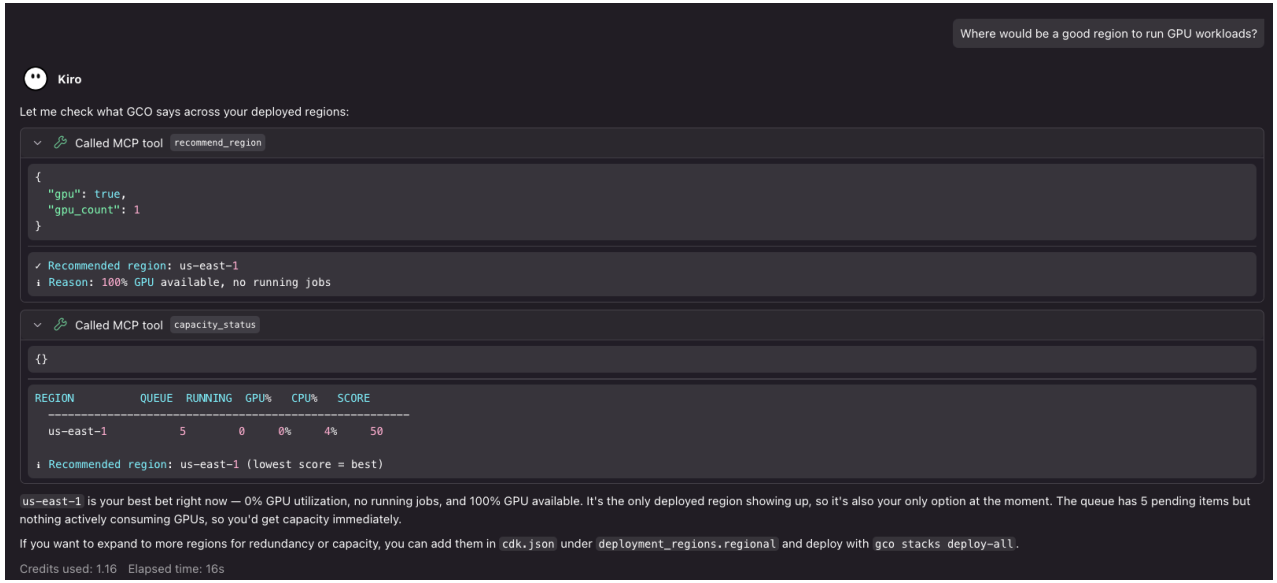




The GCO MCP server

Manage GPU infrastructure with natural language

- Wraps the entire gco CLI as LLM-callable tools — real, audited operations against your clusters.
- **Coverage:** 95 tools by default, up to 127 with feature flags — job submission, inference, capacity, costs, and more.
- **Anywhere:** Works in any MCP-capable IDE — Kiro, Claude Desktop, Cursor, and others.
- **Context:** Indexes the whole project — docs, examples, source, manifests — so agents become experts in the codebase.
  - "What's my GPU capacity in us-east-1?" "Deploy a vLLM endpoint." "Show this month's spend."



GCO Mission

A goal-directed, autonomous iteration loop

- **You define:** Declare a directive (natural-language goal), machine-checkable success criteria, a tool allowlist, and budget caps.
- Mission iterates autonomously until it reaches a verdict — continue, adjust, complete, or terminate.
- **Five phases:** Each cycle runs Propose → Execute → Observe → Evaluate → Decide.
- **Deterministic:** Criterion evaluation, verdicts, and loop-limit arithmetic are fully deterministic.
  - Optional LLM sampling is advisory only — it shapes the next strategy, never the verdict.
- **Toggle:** Off by default — enable with GCO_ENABLE_MISSION=true. See MISSION.md.

```
export GCO_ENABLE_MISSION=true

gco mission run \
  --directive "Find documentation about inference endpoints." \
  --tool-allowlist find_examples --tool-allowlist find_docs \
  --max-iterations 5 --max-wall-clock 300
```

https://github.com/awslabs/global-capacity-orchestrator-on-aws/blob/main/docs/MISSION.md

05

Foundation and Demo

Engineering quality, operations, and a live demo

Examples library

30+ submit-ready manifests — copy-paste starting points

Getting started

Simple job, SQS submission, model download

GPU & accelerators

Multi-GPU DDP, time-slicing, EFA training, Trainium & Inferentia

Schedulers

Volcano, Kueue, YuniKorn, Slurm, Ray examples

Inference

vLLM, TGI, Triton, TorchServe, SGLang endpoints

Storage

EFS output, FSx for Lustre, Valkey, pgvector

Pipelines

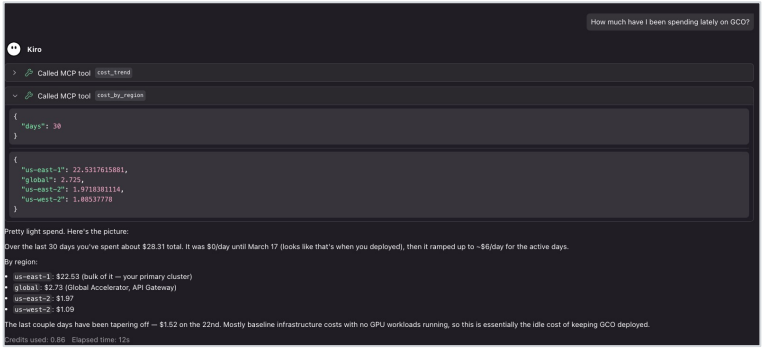
Multi-step DAGs with dependencies and failure handling

https://github.com/awslabs/global-capacity-orchestrator-on-aws/tree/main/examples

Operations: cost & monitoring

A single pane of glass across every Region

- **Monitoring:** All Regions flow into one CloudWatch dashboard, provisioned automatically by CloudFormation.
- Built-in alarms and SNS alerting; GPU metrics scraped via the NVIDIA DCGM exporter.
- **Cost tools:** Track spend by service, Region, and workload, with forecasting via Cost Explorer.
  - gco costs summary · regions · trend · workloads — all from the CLI.



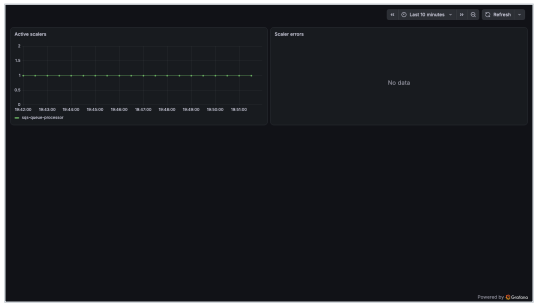
In-cluster observability with Prometheus & Grafana

kube-prometheus-stack on every regional cluster — GPU, scheduler & service dashboards, private by default

- **Per-Region stack:** kube-prometheus-stack (Prometheus, Grafana, Alertmanager, node-exporter, kube-state-metrics) ships with the regional stack — toggled in cdk.json.
- **GPU dashboards:** the NVIDIA DCGM exporter streams per-GPU utilization, framebuffer memory, temperature, and power on EKS Auto Mode nodes.
- **Auto-discovered:** ServiceMonitors scrape the schedulers (KEDA, Volcano, Kueue, KubeRay, YuniKorn); PodMonitors scrape GCO's own services.
- **Private by default:** ClusterIP-only, gp3-backed persistence, auto-rotated Grafana admin — reach it with gco monitoring open.

GPU · DCGM

Schedulers & Queues



KEDA Autoscaling

GCO Services

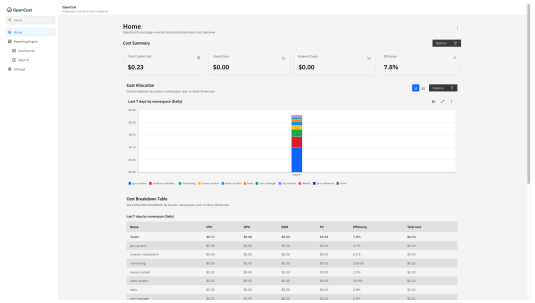
Deep dive: docs/MONITORING.md

Cost monitoring & cost-aware scheduling

OpenCost per Region, central S3 + Athena analytics, and spot price-gated dispatch — private by default

- **Per-Region OpenCost:** meters every namespace and workload from the in-cluster Prometheus; powers the GCO Cost Grafana dashboard and the native OpenCost UI.
- **Central S3 + Athena:** hourly Parquet cost reports from every Region land in one bucket; query cross-Region with gco costs k8s namespaces · regions · trend · top.
- **Cost API:** ad-hoc reports on demand via /api/v1/cost — gco costs report generate · status — IAM-authenticated like every GCO endpoint.
- **Spot price-aware scheduling:** queue jobs carry --max-spot-price; the central queue defers dispatch while the live EC2 spot price is above the cap.
- **One toggle:** cost_monitoring in cdk.json, riding the cluster-observability stack; off cleanly when disabled.

GCO Cost (OpenCost) Grafana dashboard



Native OpenCost UI — allocation & efficiency

Reach them over the same private SSM path as Grafana — no public exposure:

gco costs dashboard --region us-east-1 --via-ssm auto → localhost:3000 (Grafana)
gco costs dashboard --service opencost --via-ssm auto → localhost:9091 (OpenCost UI)
gco costs report generate · gco costs k8s namespaces → S3 Parquet + Athena

Deep dive: docs/COST_MONITORING.md

Built on a serious engineering foundation

Quality and security baked into CI

90%>	9,000+	10 + 9	32
unit-test coverage; strict floor enforced in CI	unit tests (PyTest + BATS)	security scanners + linters	CDK synth matrix configs

- All scanners and linters run in strict mode; every exception is logged in git with a compensating control.
- Scanners: Bandit, CodeQL, Trivy, Checkov, KICS, Semgrep, Gitleaks, Trufflehog, pip-audit, NPM audit.
- Compliance validated with CDK-nag against AWS Solutions, Serverless, HIPAA, NIST 800-53, and PCI DSS compliance packs.

Live demo

Try it for yourself — paste this into an MCP-capable IDE (Warning: this will run a compute job that will incur AWS costs):

> “I'm new to GCO and I want to run an LLM. Show me my options and why I'd use this instead of just spinning up my own GPU box.
>
> Pretend I'm skeptical. Convince me GCO is worth it for multi-region GPU inference — and back it up using the actual project, not marketing.
>
> Then write a manifest that calculates the value of PI, run it and print the logs using the MCP. Afterwards clean up the workspace and delete the new manifest.”

Thank you

One API. Every Accelerator. Any Region.

Jacob Mevorach · Senior GenAI Solutions Architect · Frontier AI Team

github.com/awslabs/global-capacity-orchestrator-on-aws

Docs: README · QUICKSTART · ARCHITECTURE · CLI · INFERENCE · ANALYTICS · MISSION

aws